This is a different way to test for nested elements. The original assert_select from the controller test, which was "#project_#{@project.id} .name", could instead be written as follows:

```
assert_select("#project_#{@project.id}") do |matches|
  matches.each do |element|
    assert_select(element, ".name")
  end
end
```

This may be more explicit in some cases or may allow for more complex logic for the internal selector.

The inner assert_select in that snippet is doing something you haven't seen. The first argument, element, is one of the matching HTML node objects. When assert_select is passed an HTML node, it searches that HTML for the matching selector rather than the default behavior, which is to use the current HTTP response object.

That first argument is handy for the block syntax, but you can also take advantage of it to use assert_select against arbitrary strings that happen to be valid HTML with a parent root element. All you need is a helper method, which you'd place in your test_helper.rb file:

minitest/02/test/test_helper.rb

```
require File.expand_path("../../config/environment", __FILE__)
require "rails/test_help"
require "mocha/mini_test"

module ActiveSupport
  class TestCase
    # Setup all fixtures in test/fixtures/*.yml for all tests in alphabetical
    # order.
    fixtures :all

    # Add more helper methods to be used by all tests here...
  end
end

def assert_select_string(string, *selectors, &block)
  doc_root = Nokogiri::HTML::Document.parse(string).root
  assert_select(doc_root, *selectors, &block)
end
```

You're using Rails HTML parsing classes, which aren't normally part of a balanced Rails breakfast but allow you to parse a string into an HTML document and take the root element. That root element can then be used as the basis for an assert_select search. You'd then use this method in your helper test.